

Technikai interjú felkészítő

Software Architect — API & Data Exchange Platform · .NET / Kubernetes / REST

FELTETT az interjún ténylegesen elhangzott · **VÁRHATÓ** kézenfekvő follow-up ugyanabban a témában

1 · REST

K Mi az a REST? **FELTETT**

A REST (Representational State Transfer) egy **architekturális stílus** elosztott rendszerekhez, amit megszorítások (constraints) halmaza definiál, nem pedig egy protokoll. Egy rendszer akkor RESTful, ha teljesíti őket:

- **Client-server** — felelősségek szétválasztása; a UI és az adattárolás egymástól függetlenül fejlődik.
- **Stateless** (állapotmentes) — minden kérés magával viszi a szükséges kontextust; a szerver nem tárol kliens-session állapotot a kérések között.
- **Cacheable** — a válaszok jelzik, hogy cache-elhetők-e.
- **Uniform interface** (egységes interfész) — erőforrások URI-kkal azonosítva, reprezentációkon keresztül manipulálva, önleíró üzenetek és HATEOAS.
- **Layered system** (rétegzett) — a kliens nem tudja megmondani, hogy közvetlenül az origin szerverrel vagy egy közbülső elemmel (proxy, gateway) beszél.
- **Code on demand** (opcionális).

Az erőforrások főnevek (`/orders/42`), és a HTTP igék fejezik ki a műveletet: **GET** olvasás, **POST** létrehozás, **PUT** csere, **PATCH** részleges módosítás, **DELETE** törlés.

Egymondatos: „A REST egy állapotmentes, erőforrás-orientált architekturális stílus HTTP felett, ahol az erőforrásoknak URI-juk van, és a szabványos igék definiálják a műveleteket — nem protokoll és nem framework.”

K REST vs. RESTful és a Richardson Maturity Model

VÁRHATÓ

Sok „REST”-nek nevezett API valójában RPC HTTP felett. A **Richardson Maturity Model** osztályozza, mennyire RESTful egy API a valóságban:

- **Level 0** — egyetlen endpoint, a HTTP csak alagút (sima RPC).
- **Level 1** — külön erőforrások, de az igék még az URL-ben.
- **Level 2** — rendes HTTP igék + státuszkódok (a legtöbb „REST” API itt tart).
- **Level 3** — HATEOAS: a válaszok linkeket tartalmaznak a kapcsolódó műveletekhez/erőforrásokhoz.

K Idempotencia & safe metódusok

VÁRHATÓ

Safe = nincs állapotváltozás (**GET** , **HEAD** , **OPTIONS**). **Idempotens** = ugyanaz az eredmény, akár egyszer, akár N-szer hívod.

Metódus	Safe	Idempotens
GET	Igen	Igen
PUT	Nem	Igen
DELETE	Nem	Igen
POST	Nem	Nem
PATCH	Nem	Nem feltétlenül

Gyakorlati megközelítés: hogy a **POST** hálózati hiba esetén újrapróbálható (retry-safe) legyen, használj **idempotency key**-t, hogy a szerver deduplikálja az ismételt beküldéseket.

2 · API verziózás

K Hogyan lehet egy REST API-t verziózni?

FELTETT

Négy elterjedt stratégia, nagyjából népszerűségi sorrendben:

Stratégia	Példa	Trade-off
URI path	<code>/v2/orders</code>	Legláthatóbb, könnyű routing/cache; „tisztátalan” (verzió az erőforrás-azonosítóban)

Query param	<code>/orders?v=2</code>	Könnyű default; ronthatja a cache-elést
Custom header	<code>X-API-Version: 2</code>	Tiszta URL-ek; kevésbé felfedezhető, böngészőből nehezebb tesztelni
Media type (content negotiation)	<code>Accept: application/vnd.company.v2+json</code>	Leginkább „RESTful”; legösszetettebb, legkevésbé intuitív

Amit hangsúlyozz: a verziózás célja a kontraktus fejlesztése úgy, hogy a *meglévő fogyasztók ne törjenek el*. Mondj preferenciát (az URI-verziózás a pragmatikus default publikus API-khoz; header/media-type a szigorúbb REST-tisztasághoz) és indokold — az interjúztatót az érvelés érdekli, nem a „helyes” válasz.

K Hogyan tartod karbantarthatónak a verziókat? VÁRHATÓ

- A **kontraktust** verziózd, ne minden belső változást. Az additív változásokhoz nem kell új verzió.
- Preferáld az **expand/contract (parallel change)** mintát: add hozzá az új mezőt/endpointot, migráld a fogyasztókat, majd távolítsd el a régít — a kemény cutover helyett.
- Publikálj **deprecation policy**-t (sunset dátumok, `Deprecation` / `Sunset` headerek), hogy a fogyasztók figyelmeztetést kapjanak.
- Szemantikus verziózás logikája: **major** = breaking, **minor** = additív/visszafelé kompatibilis, **patch** = javítás.

3 · Breaking change

K Mi az a breaking change, és hogyan kapcsolódik a verziózáshoz? FELTETT

A **breaking change** a kontraktus bármely olyan módosítása, ami arra kényszeríti a meglévő fogyasztókat, hogy módosítsák a kódjukat a további működéshez. Ez váltja ki az új **major** verziót.

Breaking (új major verzió kell)

- Mező, endpoint vagy paraméter eltávolítása vagy átnevezése.
- Opcionális kérés-mező kötelezővé tétele.
- Egy mező típusának vagy szemantikus jelentésének megváltoztatása.
- Validáció szigorítása (korábban elfogadott input elutasítása).
- Válasz-struktúra, hibakódok vagy alapértelmezett viselkedés megváltoztatása, amire a fogyasztók építettek.
- Olyan enum-érték eltávolítása, amit a kliensek küldhetnek.

Non-breaking (visszafelé kompatibilis)

- Új endpoint hozzáadása.
- *Opcionális* kérés-mező hozzáadása.
- Új mező hozzáadása a válaszhoz, **ha** a kliensek ignorálják az ismeretlen mezőket (tolerant reader).
- Új *opcionális* HTTP header hozzáadása.

Kulcselv — Robustness / Tolerant Reader: „Legyél konzervatív abban, amit küldesz, és liberális abban, amit elfogadsz.” A fogyasztók ignorálják az ismeretlen mezőket, hogy az additív változások non-breaking maradjanak. Hogy egy változás breaking-e, részben azon múlik, mennyire toleránsak a klienseid.

4 · Enum bővítése

K Breaking change-e egy új érték hozzáadása egy enumhoz?

FELTETT

Iránytól függ — az őszinte, senior válasz az, hogy „attól függ, és íme mitől”.

Az enum itt jelenik meg...	Egy érték hozzáadása...
Response (szerver → kliens)	Potenciálisan breaking — egy kimerítő switch-csel vagy szigorú deserializerral rendelkező kliens elszállhat vagy rosszul kezelheti az ismeretlen értéket.

Request (kliens
→ szerver)

Általában **non-breaking** — a meglévő kliensek egyszerűen sosem küldik az új értéket; az újak opt-inelnek.

Miért kockázatos a response-oldal:

- Szigorú enum-deszerializáció (pl. ismeretlen névre hibázó) parse-időben dob kivételt.
- Default nélküli kimerítő **switch** csendben átugorja az új esetet.
- A kliensoldali validáció simán elutasíthatja a payloadot.

Hogyan tedd az enumokat bővíthetővé:

- Tervezd a klienseket **tolerant reader**-nek: mindig legyen **default** / „unknown” ág.
- Dokumentáld, hogy az enumok **nyitottak** (bővíülhetnek) vagy **zártak** (fix halmaz) — a kontraktus részeként.
- **.NET**-ben: a nem mappelt értékek alapból az alapul szolgáló egészre deszerializálódnak (ez gond, ha a kód csak a nevesített tagokat feltételezi), ezért védekezz **Enum.IsDefined**-dal vagy fallback taggal.
- Fontold meg egy fenntartott **UNKNOWN = 0** tag használatát (Protobuf/gRPC konvenció), hogy a beállítatlan/ismeretlen érték biztonságos defaultra mappelődjön.

Tömör válasz: „Egy enum-érték hozzáadása requestben non-breaking, de responseban potenciálisan breaking, mert a meglévő kliensek nem feltétlenül kezelik az ismeretlen értéket. Biztonságos, ha a kliensek tolerant readerként, default ággal íródtak — szóval ez valójában kontraktus-tervezési kérdés.”

5 · CQRS

K Mi az a CQRS, és milyen előnyt ad? **FELTETT**

CQRS = Command Query Responsibility Segregation. A modellt két útra bontja:

- **Command** (parancs) — állapotot változtat, ideálisan nem ad vissza adatot. Az írási oldal.

- **Query** (lekérdezés) — adatot ad vissza, sosem változtat állapotot. Az olvasási oldal.

Egyetlen, mindkettőt kiszolgáló modell helyett van egy **write model** és egy **read model**, mindegyik a saját feladatára optimalizálva.

Előnyök

- **Független optimalizáció** — az írási oldal invariánsokat és konzisztenciát tart be; az olvasási oldal a gyors lekérdezésre van szabva (denormalizált, előre kiszámolt view-k).
- **Független skálázás** — az olvasások általában jóval túlszárnyalják az írásokat; skálázd őket külön.
- **Eltérő tárolás oldalanként** — pl. normalizált SQL write store és denormalizált/NoSQL vagy keresés-optimalizált read store.
- **Egyszerűbb modellek** — mindegyik oldal elkerüli egy közös, mindent tudó modell kompromisszumait.
- **Biztonság/tisztaság** — az írási aggályok (validáció, jogosultság) elkülönülnek az olvasási aggályoktól.

Költségek (ezeket említsd — kiegyensúlyozottságot mutat)

- Extra komplexitás; egyszerű CRUD-hoz nem indokolt.
- Ha a read és write store külön van, öröklöd az **eventual consistency**-t — az olvasási oldal lemarad az írástól.
- Több mozgó alkatrész, amit szinkronban kell tartani (gyakran események/projekciók révén).

Gyakori félreértések, amiket korrigálj:

- A CQRS **nem** igényel Event Sourcingot — jól passzolnak, de függetlenek.
- A CQRS **nem** ír elő két adatbázist — alkalmazhatod csak külön read/write modellként egyetlen store-ban is.

K CQRS vs. REST — alternatívák egymásnak?

FELTETT

Nem alternatívák — különböző szinteken működnek.

- A **REST** egy *külső* architektúrális stílus: hogyan teszed ki az erőforrásokat és műveleteket egy interfészen (általában HTTP) keresztül.
- A **CQRS** egy *belső* tervezési minta: hogyan strukturárod a command vs. query kezelést az interfész mögött.

Egy REST API ülhet egy CQRS rendszer tetején: egy **POST / PUT** parancsra képződik le, egy **GET** lekérdezésre. A kliens sima RESTet lát; a CQRS implementációs részlet.

6 · Kubernetes & konténerek

K Mi a különbség pod és konténer között?

FELTETT

A **konténer** egyetlen izolált, futtatható egység — app + függőségek egybecsomagolva (pl. egy Docker image processzként futva).

A **pod** a Kubernetes **legkisebb deployolható egysége**, és egy vagy több konténert csomagol össze, amelyek együtt ütemeződnek és megosztják:

- **Hálózat** — egy IP podonként; a konténerek **localhost**-on érik el egymást.
- **Tárolás** — a podba mountolt megosztott volume-ok.
- **Életciklus** — ugyanazon a node-on együtt élnek, együtt jönnek létre/ pusztulnak el.

A Kubernetes sosem ütemez csupasz konténert — mindig podot ütemez.

Általában egy konténer podonként; több konténer csak akkor, ha szorosan összefüggenek (pl. **sidecar** loggoláshoz, egy proxy vagy egy init helper).

Egymondatos: „A konténer egyetlen becsomagolt processz; a pod a Kubernetes burok egy vagy több konténer körül, amelyek megosztott hálózati namespace-en, tárolón és életcikluson osztoznak. A Kubernetes podokat ütemez, nem konténereket.”

K Kapcsolódó K8s objektumok, amiket érdemes ismerni VÁRHATÓ

Objektum	Cél
Deployment	Deklaratív kívánt állapot podokhoz; ReplicaSeteket kezel, rolling update, rollback.
ReplicaSet	N replikát tart futásban egy podból.
Service	Stabil hálózati endpoint + terheléelosztás podok közt (a podok efemerek, az IP-k változnak).
Ingress	HTTP(S) routing a klaszteren kívülről a Service-ekhez.
ConfigMap / Secret	Kiszervezett konfiguráció és érzékeny adatok.
Namespace	A klaszter-erőforrások logikai partíciója.
StatefulSet	Stabil identitású/tárolójú podok (adatbázisok stb.).

K Konténer vs. VM VÁRHATÓ

A konténerek osztoznak a host kernelen és processz szinten izolálnak — könnyűsúlyú, gyors indulás, kevés overhead. A VM-ek hardvert virtualizálnak és teljes guest OS-t futtatnak — erősebb izoláció, nehezebb. A Docker csomagolja a konténer image-et; a Kubernetes sok konténert orkesztrál egy klaszter felett.

7 · Architektúra-elvek (a JD-ből)

A pozíció kifejezetten felsorolja ezeket — számíts rá, hogy legalább egy előjön.

K SOLID VÁRHATÓ

- **Single Responsibility** — egy osztálynak egy oka legyen a változásra.
- **Open/Closed** — nyitott a bővítésre, zárt a módosításra.
- **Liskov Substitution** — az altípusoknak használhatónak kell lenniük ott, ahol a bázistípust várják.
- **Interface Segregation** — sok kicsi, fókuszált interfész egy kövér helyett.
- **Dependency Inversion** — absztrakciókra támaszkodj, ne konkrét implementációkra.

K Domain-Driven Design (DDD) VÁRHATÓ

- **Ubiquitous language** — közös nyelv fejlesztők és domain-szakértők közt.
- **Bounded context** — explicit határ, amelyen belül egy modell konzisztens (közvetlenül a mikroszerviz-határokhöz köt).
- Építőelemek: **Entity** (identitás), **Value Object** (attribútumok definiálják), **Aggregate** (konzisztencia-határ egy root-tal), **Repository**, **Domain Event**.

K Conway törvénye VÁRHATÓ

„A szervezetek olyan rendszereket terveznek, amelyek a saját kommunikációs struktúrájukat tükrözik." Gyakorlati tanulság: a rendszerarchitektúra és a csapatstruktúra alakítják egymást — ha moduláris architektúrát akarsz (pl. mikroszervizek), igazítsd a csapathatárokat a szervizhatárokhöz (az **Inverse Conway Maneuver**). A JD név szerint kiemeli.

K TDD VÁRHATÓ

- **Red → Green → Refactor**: írd egy bukó tesztet, tedd minimálisan zölddé, majd takarítsd fel.
- Előnyök: tervezési nyomás a tesztelhető egységek felé, élő dokumentáció, regressziós biztonság.
- Teszt-piramis: sok unit teszt, kevesebb integrációs, legkevesebb end-to-end.

K Mikroszerviz trade-offok VÁRHATÓ

- **Mellette**: független deploy/skálázás, csapat-autonómia, technológiai heterogenitás, hibaizoláció.
- **Ellene**: elosztott rendszer komplexitása, hálózati latencia, adatkonzisztencia szervizek közt, üzemeltetési overhead, nehezebb debug.
- Szervizek közti konzisztencia: **Saga** minta (choreography vs. orchestration); kerülj az elosztott tranzakciókat.

8 · SQL & NoSQL

K SQL vs. NoSQL — mikor melyik? VÁRHATÓ

	SQL (relációs)	NoSQL
Séma	Fix, kikényszerített	Rugalmas / sémátlan
Konzisztencia	Erős (ACID)	Gyakran eventual (BASE)
Skálázás	Vertikális (jellemzően)	Horizontálisan barátságos
Mire jó	Relációk, tranzakciók, integritás	Nagy volumen, rugalmas/változatos adat, skála
Típusok	—	Document, key-value, column, graph

ACID = Atomicity, Consistency, Isolation, Durability. Ismerd ezeket a betűket — természetesen párosulnak a CQRS/konzisztencia témával. A JD az SQL-t követelményként, a NoSQL-t asset-ként sorolja, szóval horgonyozz a relációsra, és említsd, hol illik be a NoSQL (pl. egy denormalizált CQRS read store).

9 · .NET stack

K Várható .NET beszédpontok VÁRHATÓ

Megjegyzés: a JD „excellent knowledge of .NET stack”-et kér. Legyél kész legalább fogalmi szinten beszélni róla. Hídként: az *architektúra*-konceptiók (REST, CQRS, DI, rétegzés) közvetlenül átvihetők; a .NET-specifikumok a fölé tett szókincs-réteg.

- **ASP.NET Core** — cross-platform framework REST API-khoz; middleware pipeline; beépített **dependency injection** (közvetlenül a SOLID D-jéhez köt).
- **Minimal API vs. Controller** — két stílus az endpointok definiálására.
- **Entity Framework Core** — ORM; `DbContext`, migrációk, LINQ lekérdezések.

- **MediatR** — elterjedt library a CQRS/mediator megvalósítására .NET-ben (command/query küldése handlereknek).
- **async/await** — **Task**-alapú aszinkronitás az egész stacken; fontos a skálázható I/O-hoz.
- **Model binding & validáció** — data annotations / FluentValidation.
- **System.Text.Json** — szerializáció; releváns a fenti enum-kezelés témához.

Villámkérdés puska

Felvetés	Horgony-válasz
REST egy mondatban	Állapotmentes, erőforrás-orientált stílus HTTP felett; igék = műveletek.
Hogyan verziózz	URI / query / header / media-type; cél = fejlődés a fogyasztók törése nélkül.
Breaking change	A fogyasztókat kódmódosításra kényszeríti → új major verzió.
Enum + új érték	Requestben non-breaking, responseban potenciálisan breaking.
Tolerant reader	Ismeretlen mezők ignorálása; mindig legyen default enum-ág.
CQRS	Külön read/write modell; mindegyik külön optimalizálva & skálázva.
CQRS vs REST	Belső minta vs külső stílus — a REST ülhet CQRS-en.
Pod vs konténer	A pod konténer(ek)e)t csomagol, hálózat/tároló/életciklus közös; K8s podot ütemez.
Idempotens igék	GET, PUT, DELETE igen; POST nem.
Bounded context	DDD modell-konzisztencia határa $\approx$ szervizhatár.
Conway törvénye	A rendszerstruktúra a szervezeti kommunikációt tükrözi.
ACID	Atomicity, Consistency, Isolation, Durability.